7: @RequestBody :- Bind the body of HTTP request (typically Json) to Controller method parameter (Java object)

8:- ResponseEntity :- It represent the Entire HTTP response, Header, status, response body etc.

Lecture 5

Spring Boot Bean and it's Lifecycle

What is bean?

- In ~~Sip~~ Simple term, Bean is a java object, which in managed by Spring container (also know as IOC Container)

* IOC Container -> Contains all the beans which get created and also manage them.

How to create a Bean?

- @Component Annotation
- @Bean Annotation

Concept && Coding

1:- Using @Component Annotation :-

- @Component :- annotation follows "Convention over Configuration" approach.

- Means Spring boot will try to auto Configure based on conventions reducing the need for explicit Configuration.

- @Controller, @Service etc. all are internally tells spring to create bean and manage it.

* So here Spring boot will internally call new User() to create an instance of this class

```
@Component
public class User {

    String username;
    String email;

    // Getter
    // Setter
    // NoArgConstructor
    // All ArgConstructor
}
```

Call → User( )
{
}

* Without makeing default Constructor it haveniot work.

Resion :- Besause Spring boot does not know what to pass in these Constructor parametes.

Concept && Coding

So what to do?

@Bean comes into the picture, where we provid the external Configuration details and tells Spring boot to use it while creating a Bean.

* If we add 2 times in Configuration file, Spring will Created 2 beans of it.

Qu 1 :- How Spring boot find these Beans?
Qu 2 :- At what time, these beans get created?

Q How Spring boot find these Beans?

1 :- Using @ComponentScan annotation, it will scan the specified package and sub-package for classes annotated with @Component, @Service etc.

2 :- Through Explicit defining of bean via @Bean annotation in @Configuration class.

Concept && Coding

At What time, these beans get created

Eagerness

-> Some Beans get created, When we start up an application

-> For ex: Beans with Singleton Scope are Eagerly initialized

Lazy

-> Some Beans get created Lazily, means when they actually needed.

-> For ex: Beans with Scope like Prototype etc. are Lazily initialized

or

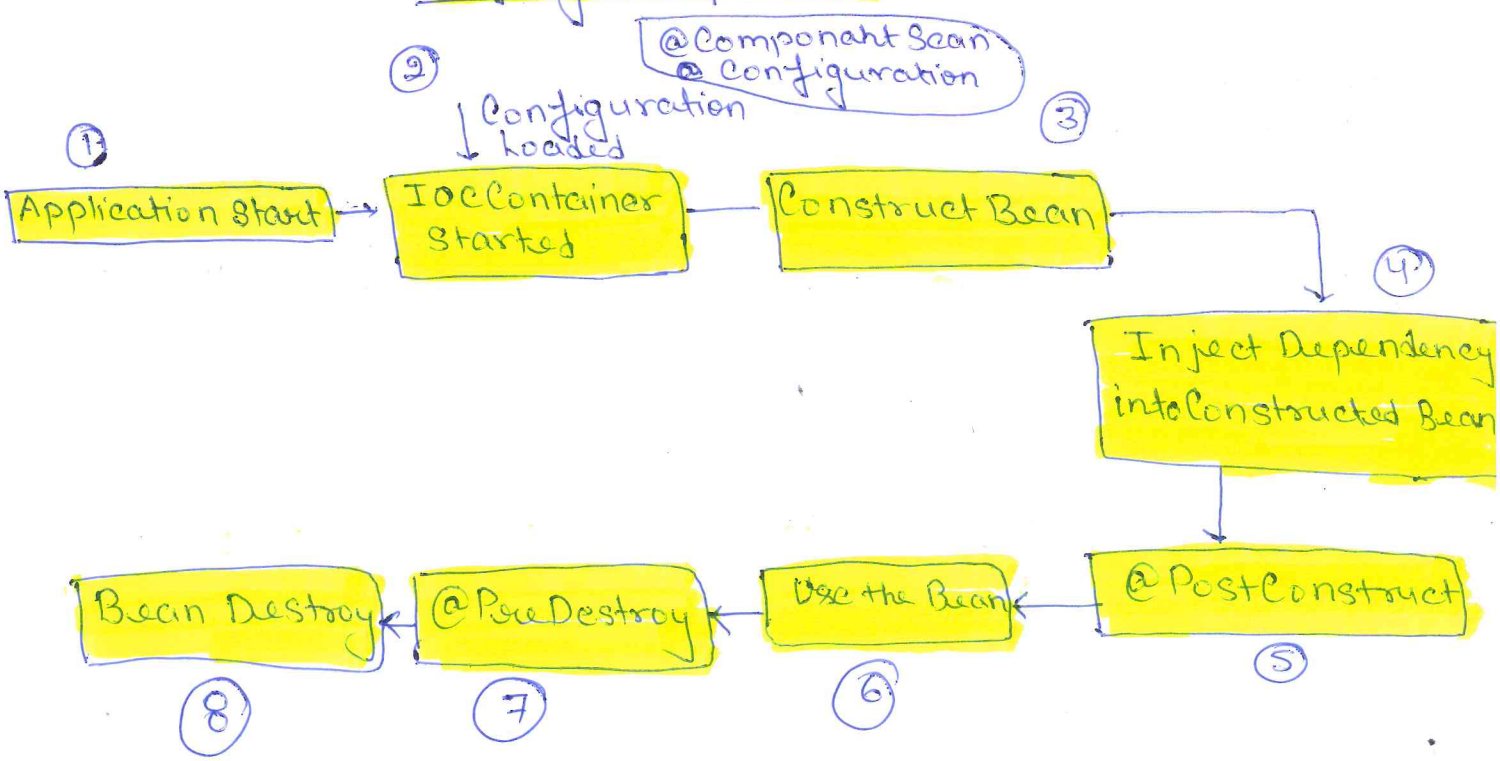
Beans with @Lazy annotation.

```
@Lazy
@Component
Public class Order {

    Public Order() {
        S.o.p("Order);
    }
}
```

Concept && Coding

Lifecycle of Bean



Step 1 :- During Application Startup, Spring boot invokes IOC Container.

(ApplicationCotext provides the implementation of IOC Container)

-> IOC Container, make use of @Configuration and @Component-Scan to look out for the classes for which beans need to be Created.

Step 2 :- Construct the Beans

```
@Component
Public Class User{
    public User(){
}
```

Step 3 :- -> Inject the Dependency into the Constructed Bean

-> @Autowired, First look for a bean of the required type.

⇒ If bean found, spring will inject it. Different way of Injection:

1. Constructor Injection
2. Setter Injection
3. Field Injection

Concept && Coding

```
@Component
Public class User{

  @Autowire
    Order order;

  Public User(){
    S.o.p("Initializing user");
  }
}
```

```
@Lazy
@Component
Public class Order{

  Public Order(){
    S.o.p("Lazy: initializing Order");
  }
}
```

Step 4: Perform any task before Bean to be used in application.

```
@Component
Public class User{

  @Autowired
   Order order;

  @PostConstruct
  Public void initialize(){
    S.o.p("Bean has been constructed and dependencies
                                have been injected");
  }

  Public User(){
    S.o.p("initializing User");
  }
}
```

Concept && Coding

Step 5 :- Use the Bean in your application

Means, we are using the bean (or Object) to invoke some methods to perform some bussiness logic.

Step 6 : Perform any task before Bean is getting destroyed.

```
@Component
Public class User{

  @PostConstruct
   Public void initialize(){ S.O.P("Post Construct initiated");

  @PreDestroy
  Public void preDestroy(){S.O.P("Bean is about to destory,
                                  in PreDestroyMethod");}

  Public User(){ S.O.P("initializing user");}

}
```

Concept && Coding